



Towards structured semantic representation via hierarchical transformer network for software defect prediction

Hongdou He ^{a,b}, Huazhi Xu ^a, Guyu Zhao ^{a,*}, Guoyan Huang ^{a,b}, Bowen Zhao ^a

^a School of Information Science and Engineering, Yanshan University, 066000, Qinhuangdao, Hebei, China

^b System Integration of Hebei Province, The Key Laboratory for Computer Virtual Technology, 066000, Qinhuangdao, Hebei, China

ARTICLE INFO

Dr. Dario Di Nucci

Keywords:

Defect prediction
Structured representation
Attention mechanism
Transformer

ABSTRACT

In recent years, deep learning techniques have gained widespread adoption in source code defect prediction. Nevertheless, significant challenges persist in effectively capturing and representing the structural characteristics inherent in programming code. Primary limitations include: (i) text sequence-based models frequently inadequately represent the distinctive structural properties of source code, and (ii) approaches dependent on compiler-generated structural representations exhibit substantial toolchain dependencies. This makes them difficult to apply to languages lacking mature compilers or to non-standard code samples. This paper addresses these challenges by systematically analyzing the fundamental differences between source code and natural language text. We propose a compiler-independent hierarchical representation method for source code. Additionally, we introduce an adaptive, layered defect prediction model named Hierarchical Transformer Network, which learns structured contextual semantic information for function-level defect detection. Experimental results on two widely used real-world datasets, CWE and CodeXGLUE, show that our method outperforms baseline models. Specifically, we observe improvements of 5.11 % and 8.58 % in AUC-ROC, respectively. Compared to the standard Transformer model, our method significantly reduces the computational resources required. The unique two-dimensional hierarchical structure enables the model to capture longer contexts with fewer resources. This approach greatly lowers the computational and learning costs for both model usage and research.

1. Introduction

Contemporary society exhibits profound dependence on complex software systems spanning diverse application domains. This pervasive reliance, however, introduces substantial security vulnerabilities and associated risks. Software vulnerabilities pose a serious threat to the digital ecosystem. The process from vulnerability activation to detection, localization, and remediation often involves substantial delays and resource consumption. Such temporal delays and resource expenditures can substantially amplify the consequential damages. For instance, in July 2024, a vulnerability in a Windows driver file from a well-known cybersecurity company led to system-wide blue screen failures in critical sectors such as banking, airports, and healthcare. These failures persisted for over half a day, causing significant economic losses and service disruptions. Notably, the rapid advancement of Artificial Intelligence (AI) technologies offers new solutions to address these challenges. AI has demonstrated great potential in software defect prediction. It

has significantly improved vulnerability detection efficiency and driven breakthroughs in related technologies.

Building upon this foundational understanding, the following section provides a comprehensive survey of contemporary approaches in software defect prediction. We systematically categorize and analyze the methodological landscape, examining both the strengths and inherent limitations of existing techniques. This analytical framework establishes the theoretical foundation for our proposed enhancements and methodological innovations.

1.1. Text-based defect prediction

Text-based approaches treat source code as a natural language sequence, convert the code text into an embedding matrix using word embedding techniques, and perform binary classification for defect detection through convolutional operations.

* Corresponding author.

E-mail addresses: hhd@ysu.edu.cn (H. He), xhh18841705273@163.com (H. Xu), gaiazhao@ysu.edu.cn (G. Zhao), hgy@ysu.edu.cn (G. Huang), 1351704387@qq.com (B. Zhao).

<https://doi.org/10.1016/j.jss.2025.112672>

Received 23 May 2025; Received in revised form 15 October 2025; Accepted 17 October 2025

Available online 22 October 2025

0164-1212/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Russell et al. (2018) treated source code fragments as one-dimensional text sequences. They employed Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) to perform defect prediction at the code block level. Le et al. (2021) combined n-gram convolution with multi-task learning to explore multi-class defect prediction. Hoang et al. (2019) input commit messages (natural language) and code changes into two separate CNNs. This process was followed by a fully connected layer to achieve defect prediction at the code change level. Gesi et al. (2021) investigated the impact of few-shot categories on model performance. They introduced a Siamese network to the DeepJIT architecture to mitigate this issue. Hoang et al. (2020) proposed CC2Vec. This model integrates attention mechanisms with RNNs to form a hierarchical attention network (HAN) that is more suitable for code structure. Building on HAN, Pornprasit and Tantithamthavorn (2022) introduced a line-level defect prediction method based on attention score ranking. Fu and Tantithamthavorn (2022) applied a BPE subword segmentation technique to reduce vocabulary size. They also used attention scores with CodeBERT for line-level defect prediction. Wartschinski et al. (2022) employed a sliding window slicing technique to create a defect prediction approach at the code segment level. This approach was specifically designed for Python code. Feng et al. (2020) jointly input natural language and code into a BERT-based architecture with a pre-training strategy for learning code representations. This approach achieved strong performance on multiple downstream tasks. Ciborowska and Damevski (2022) enhanced BERT's fully connected layers to optimize multimodal input-output mappings. This improvement increased offline indexing efficiency for defect localization. The Bugsplore model proposed by Mahbub and Rahman (2024) utilizes a RoBERTa-based pre-trained architecture. This method employs a Transformer encoder to perform bidirectional semantic representation.

This kind of approach makes it challenging to capture code-specific structural features and logical dependencies. Consequently, recent studies have increasingly incorporated structural information, such as Abstract Syntax Trees (ASTs) and Data Flow Graphs (DFGs), into training and prediction tasks related to source code (Zhou et al., 2022; Cheng et al., 2022; Mou et al., 2016). This incorporation has significantly enhanced model performance.

1.2. Structure-based defect prediction

Structure-based techniques first parse source code into structured representations such as AST and DFG, and then transform these representations into two-dimensional embedding matrices using graph convolution or structure traversal algorithms, ultimately achieving binary classification for defect detection.

Nguyen et al. (2022) proposed two different graphical source code representations using a sliding window slicing technique for module-level defect prediction. Cheng et al. (2022) constructed code paths based on ASTs. They employed an unsupervised learning method, the VPE critical path selection technique (Li et al., 2020), to select representative paths. These paths were then input into a Transformer (Vaswani et al., 2017) for training, effectively reducing the model's input size. Wu et al. (2022) assigned importance scores to statements within the Program Dependency Graph (PDG) using social network analysis. Then, a weighted code embedding matrix was constructed to ultimately facilitate module-level defect prediction through CNNs. Zhuang et al. (2021) normalized word-level PDGs to statement-level. They implemented intelligent contract defect detection with GCNs and residual connection techniques. Chakraborty et al. (2021) balanced the dataset using the SMOTE technique and trained a Gated Graph Neural Network (GGNN) with Code Property Graph (CPG) input. They designed a novel loss function during backpropagation to enhance the distinction between representations of different code categories. Zhou et al. (2019) constructed code as a CPG and employed a hybrid architecture of GCNs and GRUs to learn semantic features for defect identification. Zhang et al. (2019) integrated statement-level AST features via fully-connected layers and pooling op-

erations. These processed features were fed into a bidirectional GRU to generate code representations. Ni et al. (2020) constructed a Code Fix Tree based on ASTs and code changes. They used a Tree Convolutional Network (TBCNN) (Mou et al., 2016) to obtain tree-structured embeddings to analyze defect causes. The defect detection model proposed by Zhang and Wu (2020) adopts a Transformer-based encoder architecture. During the feature extraction stage, the AST of the source code is converted into a symbolic sequence representation. This architecture leverages the self-attention mechanism to deeply extract semantic features from the code symbol sequences, thereby achieving more accurate defect prediction.

Subsequent investigations have revealed that relying solely on a single structured input modality often fails to comprehensively capture the complete syntactic and semantic information in source code (Abdu et al., 2024). Consequently, the research community has progressively adopted multimodal input approaches to enrich the informational content provided to learning models. Zhou et al. (2022) input ASTs and CDGs (Class Dependency Graphs) into CNNs and GCNs (Graph Convolutional Networks), respectively. These processed features were then passed through fully connected layers for defect prediction at the code block level. (Hin et al., 2022) jointly input complete source code and statement slices into a hybrid Transformers and Graph Attention Networks (GATs) architecture. Through this architecture, they obtained overall and statement representations of the source code for line-level defect prediction. In a subsequent study, Hin et al. (2022) utilized multiple structured representations as input. They employed a bidirectional GRU based on attention mechanisms to construct contextual representations of statements. These contextual representations and the PDG adjacency matrix were then fed into a Graph Convolutional Network (FA-GCN). The model outputs were explained using GNNExplainer to identify defective code lines. Niu et al. (2022) jointly input code sequences, ASTs, and code descriptions into a Transformer for sequence-to-sequence pre-training. This model was fine-tuned for various downstream tasks such as code search and defect repair. Guo et al. (2020, 2022) integrated DFGs or ASTs into CodeBERT. They applied a self-attention masking mechanism to achieve directed attention interaction, while optimizing pre-training tasks for graph-enhanced BERT adaptation. The DH-CNN model, proposed by Abdu et al. (2024), employs a deep hierarchical architecture. Horizontally, it partitions code representations into syntax features based on AST and semantic features derived from control flow graphs (CFG) and data dependency graphs (DDG). Innovatively, it incorporates a gated fusion mechanism to adaptively determine the optimal combination ratio of these two types of features in the final prediction, thereby achieving a more comprehensive code representation.

These methods extract the logical structure of source code (e.g., ASTs, CPGs, etc.) as model input using compilation tools. However, ASTs and CPGs cannot be generated for all source code. Code segments with syntax errors often cause compilation failures, leading to systematic exclusion of such cases in experimental settings. Nevertheless, in practical development scenarios, developers must still assess the defect risk associated with such code. Furthermore, language-specific compilation tools are required to generate structural features. This dependency creates implementation barriers for structure-based defect prediction methods across programming languages. Therefore, these limitations necessitate a universal structured representation method. Such a method should be programming language-agnostic and defect type-independent to overcome constraints.

1.3. Other methods

Some methods for defect prediction rely on manually extracted features or non-AI approaches to detect code defects. For instance, Kamei et al. (2012) introduced "Just-In-Time" (JIT) defect prediction in 2013, which focused on defect prediction for code changes. They primarily used a set of human-defined code-related attributes as input. These

features were then processed with machine learning or deep learning techniques for defect prediction at the code change level. [Pascarella et al. \(2019\)](#) improved these manually defined metrics, making them more suitable for finer-grained defect prediction. The advantages of this approach include faster detection, smaller model parameter sizes, and easier construction. However, extracting deep-level information from the code is challenging, limiting the method's performance. Additionally, manual metrics require developers to expend considerable time constructing and collecting relevant attributes. This process significantly increases the associated costs of defect prediction. Furthermore, this approach struggles to capture the structural information of the code. Recent research has gradually integrated such methods with the strategies above to improve performance.

Another non-linguistic defect detection method involves pre-written test components to assess the robustness of the code during runtime. This method can accurately pinpoint some obvious defects but faces challenges with more sporadic defects (e.g., defects that only appear once in thousands of executions). Test components are often ineffective in detecting such rare issues.

1.4. Contributions

Despite the numerous research contributions mentioned above, several key issues remain unresolved ([Lu et al., 2021](#)). Text-sequence-based models struggle to model code structure relationships, and generating structured features through compilers presents challenges such as high complexity and poor generalization. This paper proposes a lightweight structured representation method based on source code text to address these issues. Additionally, we design a corresponding defect prediction model. The research framework consists of the following stages. First, we explore the key differences between programming languages and natural languages from the perspective of human code processing, and define a generalized structure for source code. We then extract structural information from the source code. Next, we design a corresponding defect prediction model. Finally, we conduct comparative experiments on two widely used datasets. We also examine the impact of code length and outlier data (e.g., code samples with a significantly higher line count than the average in the dataset) on model performance.

The contributions of this paper are as follows:

(1) **Lightweight Structured Representation Method:** We propose a structured representation method suitable for text-based code defect prediction models. This method applies to various programming languages, is easy to construct, and significantly reduces the cost of building code structures.

(2) **Hierarchical Transformer Network (HTN):** We design a defect prediction model (HTN) corresponding to the structured representation of code text. Compared to other text-sequence-based defect prediction models, our model learns the structural information within the code better. This enhancement improves the model's ability to capture code structure in text-based defect prediction.

(3) **Systematic Empirical Validation:** Through empirical studies, we demonstrate the effectiveness of HTN on two datasets. Our results showcase its performance improvements. We also explore the impact of varying code line truncation lengths and outlier data on model performance.

To evaluate the effectiveness of the proposed structured representation method for code text and the Hierarchical Transformer Defect Prediction Network (HTN), we conducted comparative and ablation experiments with several models on the CEW and CodeXGLUE datasets. We achieved improvements in AUC-ROC (5.11%-8.58%), Accuracy (2.55%-7.12%), Precision (4.12%-4.90%), Recall (2.51%-89.0%), and F1-Score (3.29%-42.0%) metrics. When the code line truncation length increased from 32 to 96, the model's performance improved by 8.65%. Therefore, compared to other baseline models, the defect prediction model based on our proposed structured representation method signifi-

cantly outperforms all five metrics. The code line truncation length also shows a significant impact on model performance.

The remainder of this paper is organized as follows: [Section 2](#) establishes the requisite research background and theoretical foundations; [Section 3](#) elaborates our proposed structured representation methodology and corresponding architectural framework; [Section 4](#) details the experimental design and evaluation protocol; [Section 5](#) presents comprehensive experimental results and analytical insights; and [Section 6](#) concludes with a synthesis of contributions and future research directions.

2. Approach

Motivated by the identified limitations in existing approaches, this section presents our methodological contributions. We first address the fundamental challenge of precisely characterizing contextual semantic relationships within programmatic constructs by introducing a novel structured code representation methodology. To address this limitation and derive more precise semantic representations of code defects from source code, we propose a novel structured code representation method that effectively preserves the semantic structure of programs. Building upon this framework, we develop a Hierarchical Transformer Network (HTN) designed to learn structured contextual semantic embeddings of defective code, thereby facilitating effective defect detection.

The proposed software defect detection framework, depicted in [Fig. 1](#), is divided into three parts. Subsequent sections will elaborate on this paper's core components: the structured code representation and the hierarchical defect detection network.

(1) **Code Preprocessing:** The source code undergoes systematic preprocessing involving two key operations: noise removal, which eliminates non-functional elements including code comments and other syntactically irrelevant artifacts, and normalization, which standardizes structural features such as code indent patterns and formatting conventions to ensure consistency in subsequent analysis.

(2) **Structured Code Representation:** Following preprocessing, each code segment is transformed into a code token matrix, from which semantic structural information is extracted through three distinct dimensions: Block Position, Indent Position, and Inline Position. This tri-dimensional approach precisely characterizes the contextual semantic relationships between code tokens. Then, these three positional encoding matrices are integrated with the initial embedding vector for each line of code to form the comprehensive representation.

(3) **Hierarchical Transformer Network:** We propose a hierarchical defect detection network that effectively utilizes the structured code representation described previously. The model implements a multi-level self-attention mechanism to perform bottom-up semantic embedding of source code structure, which enhances the learning of precise defect-related semantic features and enables accurate defect identification at both block and function granularities.

2.1. Structured semantic representation of code

In software defect detection, constructing accurate semantic embeddings that effectively capture defect patterns is essential for reliable defect identification. To comprehensively leverage the structured semantic information inherent in source code - including global contextual relationships, local statement scope constraints, and inline syntactic semantics - we present a novel structured code representation methodology. This approach, building upon tokenized code elements as fundamental units, systematically extracts structured descriptors that encode contextual syntax-semantic relationships. The method subsequently constructs a three-dimensional position-aware code representation, thereby establishing the groundwork for our advanced positional encoding-based hierarchical defect detection framework.

For a given function, multi-granular semantic analysis across hierarchical levels is enabled by three position-aware semantic features: global

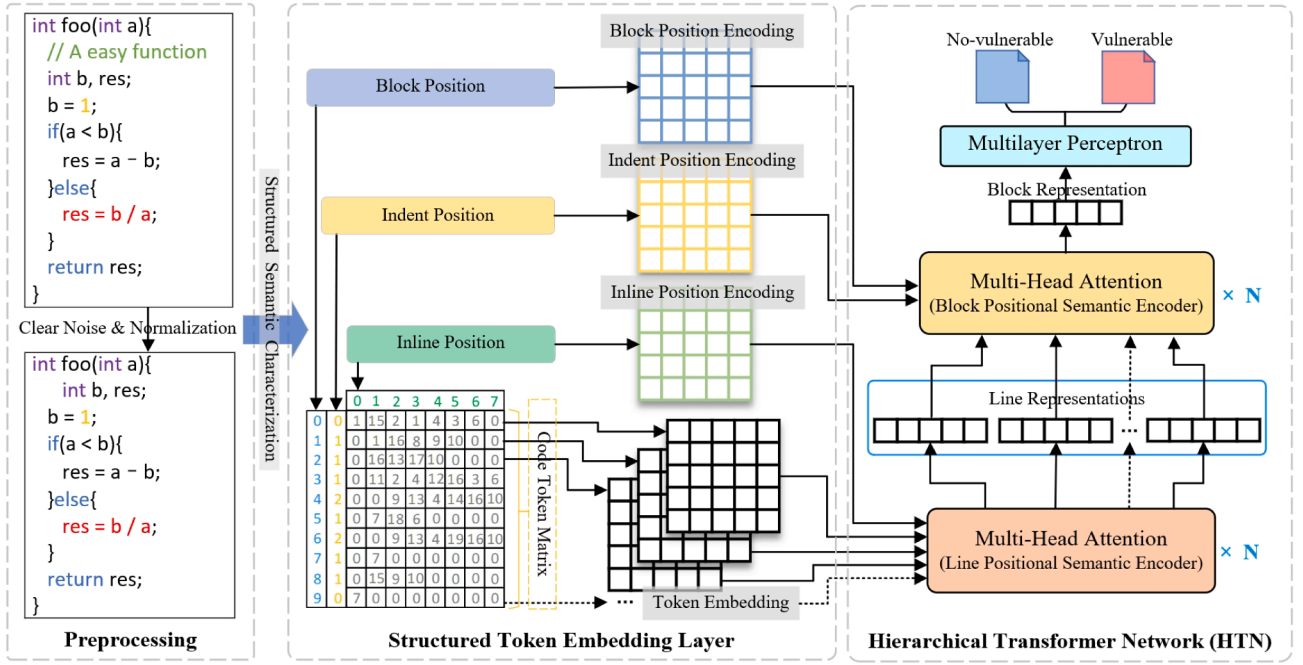


Fig. 1. The framework of our approach.

code block organization, local indent hierarchy, and inline syntactic token relationships. We propose a token-centric holistic structural representation that systematically integrates these three orthogonal dimensions: (1) *global line-level structural patterns*, (2) *local scope-based indent dependencies*, and (3) *fine-grained token sequence semantics*. As illustrated in Fig. 2 (top to bottom), this representation establishes a unified framework for encoding contextual syntax-semantic relationships through hierarchical positional awareness.

(1) **Global Statement Context** – Block Control Position: Captures high-level, coarse-grained positional semantic relationships across code lines. Represented by line numbers, it reflects execution and logic relationships. The semantic relationships between statements are often established through the contextual use of variables.

(2) **Local Scope Context** – Indent Scope Position: Represents fine-grained semantic details via the intra-line position of tokens. This captures syntactic and semantic meanings, including operators and symbols.

(3) **Inline Syntax Context** – Inline Word Position: This dimension describes the contextual positional semantic information of tokens within a line of code. It is the most fine-grained semantic information, represented by the position of tokens within a line. This position reflects the syntactic and semantic encoding of operations, including symbols.

Together, these dimensions form a structured semantic code representation. Our approach is inspired by how developers understand code: interpreting not just individual statements, but also their order and scope. Unlike natural language, code includes indentation to denote scope – an important semantic cue, particularly in languages like Python. Even in languages without mandatory indent, compilers infer such information. Hence, developers rely on these three dimensions to comprehend code logic and identify vulnerabilities.

Based on the above analysis, the positional information of each line of code is sequentially extracted from top to bottom within the code block: block-level line control positional information, line-level indent scope positional information, and inline word positional information. Subsequently, a position-based tokenization process is applied to obtain a structured lexical representation of the code, as illustrated in Fig. 3.

Each token's position in code can be abstracted as a triplet $Pos = (l, h, i)$, where

- l : the position of the token within a line (inline order);
- h : the line's position within the block (inter-line order);
- i : the indent level, indicating scope.

This allows us to obtain a detailed positional representation of the word within the code segment.

Thus, a code block or function is modeled as a matrix $W_f^{(|c| \times |l|)}$, where $|c|$ is the number of code lines, and $|l|$ is the number of tokens per line.

The source code defect detection task can be abstracted into a binary classification problem, which involves learning to predict which code blocks in the collection have defects. For a defect detection model that takes the structured semantic information of source code as input, the data samples can be defined as:

$$\{(C_k, Pos_k), Y_k\} | C_k \in C, Pos_k \in Pos, Y_k \in \{0, 1\} \quad (1)$$

where C represents the set of all source-code function texts in the dataset; $Pos = (l, h, i)$ is a triple of the structured representation of the source code constructed through calculation; Y_k is the real class label of a certain code block C_k , where 1 indicates the presence of a defect and 0 indicates no defect or no defect found. We collectively refer to the set of labels Y_k as Y , $k = 1, 2, \dots, n$ and n is the total number of functions in the dataset.

Each code block is represented as:

$$C = [(K_1, i_1), (K_2, i_2), \dots, (K_{|H|}, i_{|H|})] \quad (2)$$

where $|H|$ is the number of lines, K_i is the i th line, and i_m denotes indent.

Each line K_i is represented as:

$$K_i = [k_{i1}, k_{i2}, k_{i3}, \dots, k_{i|L|}] \quad (3)$$

where $|L|$ is the number of tokens in the line, and k_{ij}^d is the j th token in line i , with embedding dimension d . This approach transforms flat code into a three-dimensional structure, offering better representation than natural-language-like methods. Unlike ASTs, it does not require compilers or parsers, enhancing generalizability.

For each code block sample C_i in the program, the goal of source code defect detection is to learn a mapping $f : (C, Pos) \rightarrow Y$ to determine the label for each given code block. This predicted label is used to assess whether there are vulnerabilities in the code block. The mapping f is

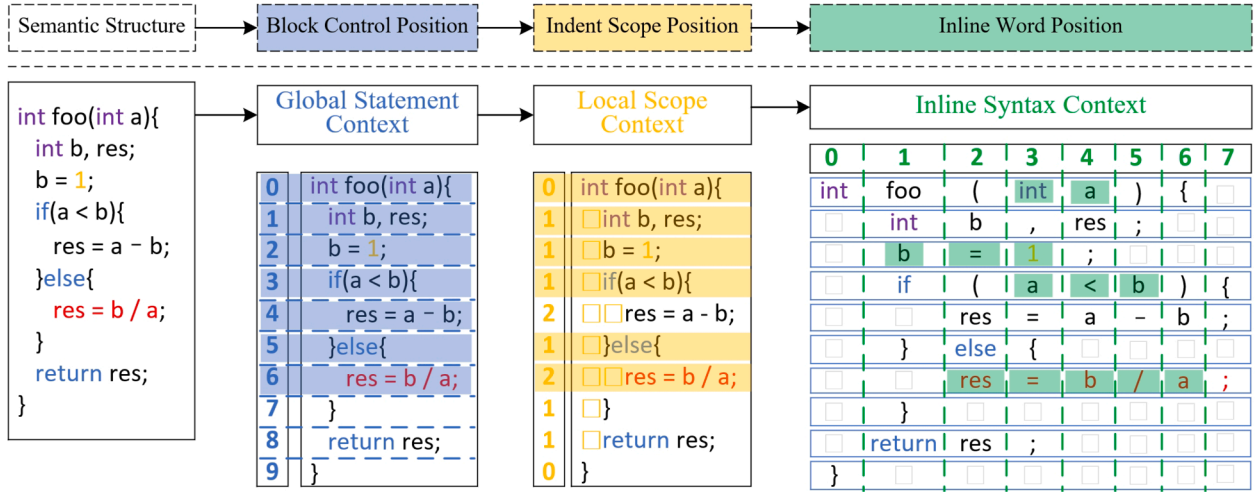


Fig. 2. Tri-dimensional structured semantic representation of source code.

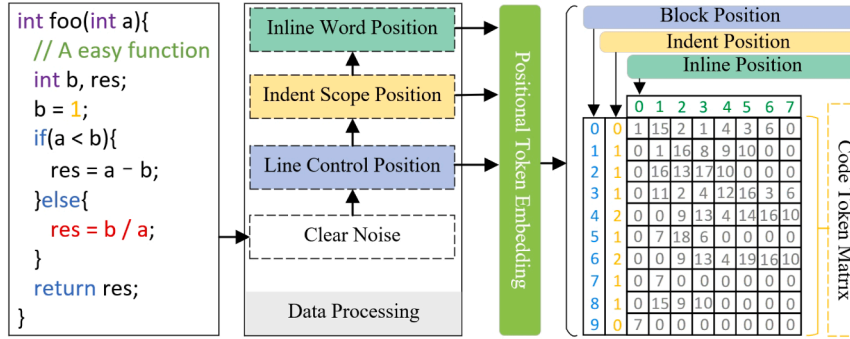


Fig. 3. Extraction and representation of structured semantic information of code.

the defect prediction function, which can be learned by minimizing the following loss function:

$$\min \sum_{k=1}^n L(f(C_k, Pos_k), Y_k) \quad (4)$$

where L is the cross-entropy loss function.

We tokenize the code into unique identifiers and initialize their embeddings randomly. Using HTN, a Line Encoder captures structured line semantics, followed by a Block Encoder for block-level structure. Training via backpropagation generates embedding matrices encoding structured positional semantics.

2.2. Hierarchical defect representation learning network

In this section, we propose a Hierarchical Transformer Network (HTN) for source code representation learning and defect detection based on the structured semantic code representation constructed above. The HTN network consists of three key modules:

- (1) A Line Encoder module for learning the structured contextual information within code lines, which generates the embedding representation for each code line;
- (2) A Block Encoder module for learning the structured contextual information between different code lines, producing the final embedding representation for code segments;
- (3) A network that uses the code block embeddings for defect detection.

Fig. 4 illustrates the details of the HTN model architecture.

HTN improves upon HAN (Hoang et al., 2020) by accommodating the three-dimensional structured semantic representation of code intro-

duced in this paper. Specifically, we incorporate a Transformer-based Positional Encoding to capture the positional semantic information accurately. This is achieved at Inline Position, Indent Position, and Block Position levels, unlike RNNs, which can only infer a node's position based on information propagated from prior cells. Transformer-based positional encoding allows for a richer perception of positional information. This flexibility is achieved through customizable encodings. This approach has also been successfully applied in other domains (Dosovitskiy et al., 2020).

HTN introduces several improvements to the Transformer architecture to better align with our approach:

- (1) We do not treat code and positional information as a one-dimensional structure (i.e., flattening the entire code segment into a single line). This decision is driven by two considerations: (i) the inherent multi-dimensionality of the code's structured representation, which would lead to redundant computation if treated as one-dimensional; (ii) flattening the code segment would result in a vast model dimension, significantly increasing both training and application costs.

- (2) To represent the constructed positional information more effectively, we leverage the flexibility of Transformer positional encoding. Specifically, we use distinct encoding methods for different types of positional information.

2.2.1. Line encoder

We utilize a Line Encoder to process each line in a code block to capture the semantic features of individual tokens within a code line and their sequential relationships. This process extracts context-aware embeddings for every line.

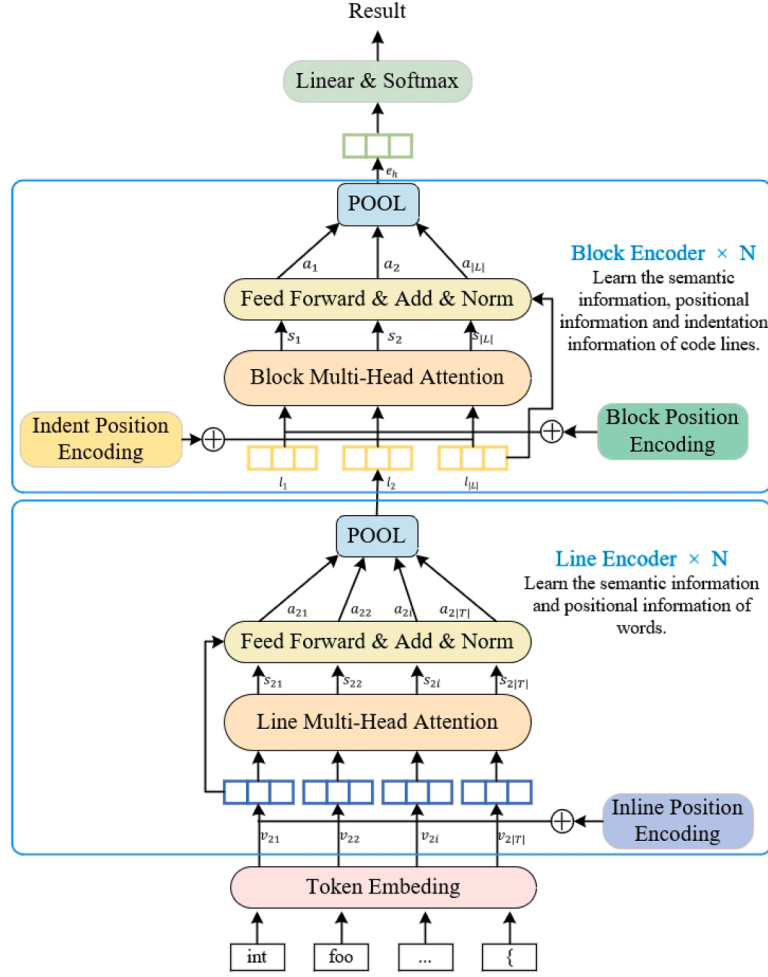


Fig. 4. The framework of hierarchical transformer network (HTN).

For the input code token representation matrix, we start by generating learnable random embeddings to expand their dimensionality. We then use the in-line token position—specifically, the element l from the positional triplet $Pos = (l, h, i)$ —and its semantic information to determine its importance in the attention mechanism. The element l represents the token's position in the code. Since there is a sequential order among the words within a single line, we use sine-cosine positional encoding to generate the embedding vector of l :

$$LPosEnc(pos, 2d) = \sin\left(\frac{pos}{1000^{\frac{2d}{d_{model}}}}\right) \quad (5)$$

$$LPosEnc(pos, 2d + 1) = \cos\left(\frac{pos}{1000^{\frac{2d}{d_{model}}}}\right) \quad (6)$$

where pos denotes the position of the word in the line, d_{model} refers to the dimensionality of the embedding vectors, and d is the d th dimension within the embedding.

The Line Encoder learns from both the positional information and the semantic content of each word within a code line, thus generating more accurate embedding representations for each line of code:

$$V_i^{|T| \times d_{model}} = Emb(K_i) \oplus LPosEnc(K_i) \quad (7)$$

where i represents the line index within the code block, $|T|$ denotes the token count of the current code line, and d_{model} indicates the embedding dimension.

The embedding matrix, which incorporates the line positional information, is processed through multi-head self-attention to produce the

attention-weighted embedding matrix for the code line, denoted as S_i :

$$S_i^{|T| \times d_{model}} = MultiHead(Q, K, V) \quad (8)$$

$$= Concat(head_1, \dots, head_h)W^o$$

Since we choose a self-attention mechanism, the matrices Q, K, V are derived from our $V_i^{|T| \times d_{model}}$ matrix. The function $Concat$ combines the attention matrices output from each head, and the average is computed. W^o is a learnable weight matrix used for linear transformation, while $head_i$ is the output from each head, calculated as follows:

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V) \quad (9)$$

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (10)$$

where W_i^Q, W_i^K, W_i^V are learnable matrices, and d_k denotes the embedding dimension. After passing through a $softmax$ layer, we obtain the output for the current attention head.

Finally, after applying residual connections and a feedforward neural network, we yield the embedding vector for each code word:

$$a_{ij} = Relu((s_{ij} \oplus v_{ij})W_1 + b_1)W_2 + b_2 \quad (11)$$

where W_1 and W_2 are learnable weight matrices, b_1 and b_2 are bias terms, and s_{ij} is a column from the code line embedding matrix S_i , representing the attention-weighted embedding of a word. The vector a_{ij} retains important information learned from the attention mechanism and initial information that facilitates further learning by the Block Encoder.

Through the self-attention mechanism, the HTN Line Encoder learns both the semantic and positional information of each token within a code line, enabling it to focus on the most significant information within the line. By employing residual connections, the output retains the positional information among code tokens, facilitating the learning process of the block encoder.

For each code line, we stack the embedding vectors a_{ij} of all tokens to form an embedding matrix A_i , which represents the entire line. We then apply a pooling operation to obtain the line-level embedding vector:

$$l_i = \text{Pool}(A_i^{[T] \times d_{\text{model}}}) \quad (12)$$

The resulting line embeddings are fed into the Block Encoder to model the structural relationships between lines further, ultimately producing a code block embedding enriched with structural information.

2.2.2. Block encoder

To capture the semantic, sequential, and indent information from diverse code line embeddings, we leverage the embedding vectors l_i generated by the line encoder. The block encoder further processes these vectors to yield a structured embedding vector for the entire code block, which is contextually associated with the source code and indent.

Thus, at this stage, we need to encode two additional positional elements:

(1) **Line Positional Encoding:** The block encoder's function enables the self-attention mechanism to focus on the sequential information among different code lines. We adopt sinusoidal encoding as the block positional encoding to represent the position of code lines.

(2) **Indent Encoding:** The indent encoder empowers the self-attention mechanism to capture the indent information between different code lines, facilitating the learning of the scope of each code line. Code indent usually indicates the scope (such as variable lifecycle and available parameters) of a code line within the entire code block rather than its sequence. Hence, we employ a learnable query embedding matrix as the encoder for the indent values.

By performing element-wise addition of the block and indent positional encodings with the embedding representation L learned by the line encoder, we obtain the embedding representation H that will be input into the block encoder:

$$H = L \oplus I\text{PosEnc}(I) \oplus L\text{PosEnc}(L) \quad (13)$$

where I is a one-dimensional array composed of the indent values for each line, and $L = \{l_1, l_2, \dots, l_{|L|}\}$ represents the combination of all code line embeddings (i.e., the matrix formed by all embeddings l_i), with $|L|$ being the number of lines in the code.

By adding the two positional encodings to the code line embeddings element-wise, we preserve the sequential and indent information within the embedding vectors. This enables the model to extract the structured semantic details of the entire code text more accurately.

Subsequently, we employ a multi-head self-attention mechanism to obtain the attention-weighted code block embedding representation matrix S :

$$S^{[L] \times d_{\text{model}}} = \text{MultiHead}(Q, K, V) \quad (14)$$

$$= \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^o$$

where d_{model} is the output dimension of the embedding representation after passing through the feed-forward neural network in the line encoder.

In contrast to one-dimensional text sequence inputs, we leverage three-dimensional positional encodings and two layers of attention mechanisms. This allows the model to learn the semantic information of the entire code block more accurately.

Finally, after applying residual connections, the feed-forward neural network, and pooling, we derive the embedding representation for each code line after the attention mechanism:

$$a_i = \text{Relu}((s_i \oplus h_i)W_1 + b_1)W_2 + b_2 \quad (15)$$

where s_i is a row from the code block embedding representation matrix S , representing the embedding vector of a code line after attention weighting; W_1 and W_2 are learnable weight matrices; and b_1 and b_2 are bias values.

By applying the same stacking and pooling method as in the line encoder, we obtain the embedding representation of the entire code block:

$$e_h = \text{Pool}(A) \quad (16)$$

Finally, the code block's embedding representation is fed into a fully connected layer for classification label prediction.

2.3. Defect prediction classifier

After aggregating information via HTN, we acquire the embedding representation e_h of the entire code block. Subsequently, we further process this embedding vector using a Multi-Layer Perceptron (MLP) (Popescu et al., 2009). This processing maps the vector into the classification confidence space for defect prediction, allowing us to determine whether the code segment has defects.

Given that the line encoder and the block encoder incorporate feed-forward neural networks, there is no need to design an overly complex fully-connected neural network for the final defect prediction stage. We transform the code line embeddings into a label for subsequent classification:

$$\hat{y} = \text{Sigmoid}(\text{MLP}(e_h)) \quad (17)$$

where, e_h is the embedding representation of the code segment obtained by aggregating source code information through the HTN. The MLP, a fully-connected neural network, undergoes two mapping transformations and can be expressed as:

$$\text{MLP}(e_h) = \text{Sigmoid}((e_h)W_1 + b_1)W_2 + b_2 \quad (18)$$

where W_1 and W_2 are learnable weight matrices that can map the embedding vector of the code segment to another dimension. Meanwhile, b_1 and b_2 are bias values used to adjust the output results, thereby further enhancing the model's expressiveness.

Finally, the *Sigmoid* function is an activation function that can map the classification results to the range of $[0,1]$:

$$S(x) = \frac{1}{1 + e^{-x}} \quad (19)$$

At this point, the final label for classification has been obtained.

In the classification task, the binary cross-entropy loss function is often the preferred choice for measuring the discrepancy between the model's prediction results and the true labels, owing to its compatibility with the *Sigmoid* and *Softmax* activation functions. Its mathematical definition is:

$$H(y, \hat{y}) = -\frac{1}{N} \sum_i [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (20)$$

where y represents the distribution of the true labels, which typically take values of 0 and 1 in the classification task. $\hat{y}$ is the probability distribution predicted by the model.

This study derives this probability distribution by passing the code block embedding representation e_h through the MLP and the *Sigmoid* activation function. During training, this loss value is used for iterative optimization of the model, thereby improving its accuracy.

3. Experimental setup

3.1. Datasets

This study evaluates the model's performance using two datasets: CWE (Russell et al., 2018) and CodeXGLUE (Lu et al., 2021). A brief description of each dataset follows.

Table 1
CWE dataset (Unbalanced).

Dataset	Negative	Positive	Rate(%)	Total
Train	954,567	64,904	6.366	1,019,471
Test	119,180	8239	6.466	127,419
Validate	119,221	8255	6.476	127,476
Total	1,192,968	81,398	6.467	1,274,366

Table 2
CWE dataset (Balanced).

Dataset	Negative	Positive	Rate(%)	Total
Train	98,856	65,904	40.0	164,760
Test	12,379	8253	40.0	20,632
Validate	12,381	8254	40.0	20,635
Total	123,616	82,411	40.0	206,027

Table 3
CodeXGLUE dataset.

Dataset	Negative	Positive	Rate(%)	Total
Train	11,836	10,018	45.841	21,854
Test	1477	1255	45.937	2732
Validate	1545	1187	43.448	2732
Total	14,858	12,460	45.611	27,318

3.1.1. CWE Dataset

The CWE dataset, developed by Russell et al. (2018), consists of code collected from open-source GitHub projects. The dataset is annotated based on the CWE Top 25 vulnerability classification standard (<https://cwe.mitre.org/top25/>). It includes many real-world code samples, effectively demonstrating the model's performance in practical scenarios. The original dataset contains 1,274,366 code samples, with 6.45 % classified as defect samples (see Table 1). Due to the highly imbalanced class distribution, random negative sampling was applied to balance the dataset. The resulting distribution is shown in Table 2.

3.1.2. Codexglue dataset

The CodeXGLUE dataset, a benchmark constructed by Lu et al. (2021), is widely used in defect prediction research (Feng et al., 2020; Guo et al., 2020; Lu et al., 2021; Tang et al., 2023). As such, it is included in our experimental data. The statistical characteristics of the dataset are presented in Table 3.

3.2. Evaluation metrics

This study employs widely used evaluation metrics for classification tasks, including Precision, Accuracy, Recall, F1-Score, and the ROC curve, to comprehensively assess the model's performance.

The basic concepts are defined as follows:

- **True Positive (TP):** The number of samples that are truly positive and correctly predicted as positive.
- **True Negative (TN):** The number of samples that are truly negative and correctly predicted as negative.
- **False Positive (FP):** The number of samples that are truly negative but incorrectly predicted as positive.
- **False Negative (FN):** The number of samples that are truly positive but incorrectly predicted as negative.

Based on these definitions, the formulas for each metric are as follows:

(1) **Accuracy:** The proportion of correctly predicted samples to the total number of samples, representing the model's overall performance. A higher accuracy indicates better performance. The formula is:

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN} \quad (21)$$

(2) **Precision:** The proportion of true positive samples among all samples predicted as positive, reflecting the model's ability to identify positive samples correctly. The formula is:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (22)$$

(3) **Recall:** The proportion of correctly predicted positive samples among all actual positive samples, reflecting the model's ability to identify positive samples. A higher recall indicates better performance in identifying positive samples. The formula is:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (23)$$

(4) **F1-Score:** The harmonic mean of Precision and Recall, providing a balanced measure of model performance. The formula is:

$$F1Score = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (24)$$

(5) **ROC-AUC:** A graphical plot of the True Positive Rate (TPR) versus the False Positive Rate (FPR). The area under the curve (AUC) is used to assess the performance of binary classification models.

3.3. Research questions

To systematically evaluate the model's performance, this study addresses the following research questions:

3.3.1. RQ1: Does the proposed source code representation method and corresponding model significantly outperform existing models?

To verify the performance of HTN, we compared it with several baseline models :

- (1) **TextCNN:** A method that applies Convolutional Neural Networks (CNN) to one-dimensional sequences, modified by Russell et al. for defect prediction (Russell et al., 2018).
- (2) **BiLSTM:** A neural network model commonly used in natural language processing, effectively learning text and context information. Russell et al. applied BiGRU to defect prediction, treating code as one-dimensional text (Russell et al., 2018).
- (3) **BiGRU:** A variant of the LSTM model commonly used in natural language processing. BiGRU is easier to train and avoids the gradient vanishing and explosion issues associated with long-term dependencies in LSTM (Russell et al., 2018; Zhang et al., 2019).
- (4) **HAN-cc2vec:** Cc2Vec is the original version of the Hierarchical Attention Network (HAN), proposed by Thong et al. It combines attention mechanisms with Recurrent Neural Networks (RNNs) to better capture the structure of code (Hoang et al., 2020).
- (5) **HAN-DL:** DeeplineDP, introduced by Pornprasit et al., is an improved version of HAN. It enhances the original model by incorporating a line-level defect prediction method that utilizes attention score ranking for more accurate defect detection (Pornprasit and Tantithamthavorn, 2022).

These five baseline models were compared on the two datasets mentioned earlier. The performance of each model was evaluated based on the five metrics introduced in Section 4.2. This comparison validates the performance of HTN.

3.3.2. RQ2: What is the contribution of each component to performance improvement?

We evaluated each module's contribution using a layered ablation study. By sequentially removing key components, such as the indentation encoder and hierarchical attention mechanism, we compared the performance of the full model with the ablated models, quantifying the contribution of each module to the five metrics.

3.3.3. RQ3: Does the presence of outliers (e.g., abnormal code length or lines) affect the model's robustness?

To investigate the impact of outliers in the dataset on model performance, as suggested by Gesi et al. (2021), we designed two experiments: (1) adjusting the input length threshold (32/64/96 lines); and (2) applying the Tukey criterion to filter out outliers, evaluating the model's performance stability under data distribution perturbations.

3.3.4. RQ4: Does HTN's hierarchical attention mechanism demonstrate significant advantages over traditional transformers in terms of computational efficiency and memory usage?

To validate the theoretical advantages of HTN in inference costs, we conducted quantitative analysis by controlling model parameters (embedding dimension of 512, 4 attention heads, and 8 attention layers) to quantify the performance improvements of HTN's hierarchical attention mechanism in multi-head self-attention memory optimization.

3.4. Hyper parameters

For the baseline models, we used the optimal parameters specified in the respective original papers for training. For HTN, we set the embedding dimensions for the line encoder and block encoder to 512, and the number of layers for both to 8. The maximum number of code lines is set to 64, with a maximum code length per line of 32. Overflow or insufficient length is handled by truncation and zero-padding. In the pooling layer, we used mean pooling. The learning rate was set between $1e-5$ and $1e-8$. We employed the Adam optimizer and cross-entropy loss function for all models for training. The implementation code for HTN, all experimental configurations, and detailed instructions for dataset acquisition used in this work are publicly available at <https://github.com/hzx-HTN/HTN-master> to facilitate reproducibility and future research.

4. Results and analysis

All experimental evaluations were performed utilizing an NVIDIA GeForce RTX 3090 GPU with 24GB memory capacity. The following sections of this chapter provide a systematic analysis of the experimental results.

4.1. Performance of HTN compared to baseline models (RQ1)

To evaluate the performance of our model, we compared it with several baseline models: (1) TextCNN, which applies Convolutional Neural Networks (CNN) to one-dimensional text sequences. (2) BiLSTM and BiGRU, two widely used neural networks in natural language processing that are also commonly employed to generate code embeddings. (3) HAN, a model that utilizes a hierarchical attention network composed of bidirectional GRU (Bi-GRU) and attention mechanisms to extract code features. The output vectors of both the original and modified code representations are combined to capture their differences. HAN has demonstrated significant advantages in code representation learning through its hierarchical attention mechanism. It has been successfully applied in various downstream tasks, such as code clone detection and defect prediction. Therefore, HAN was selected as a baseline model for comparison in the subsequent experiments.

For the CodeXGLUE dataset, the experiments followed the optimal hyperparameter settings from the original study. Given the significant class imbalance in the CWE dataset, we employed a random negative sampling strategy to adjust the defect sample ratio to 0.4, optimizing the training process. The experimental results for the CodeXGLUE and CWE datasets are presented in Figs. 5 and 6, respectively. To validate the effectiveness of the proposed method and ensure the statistical reliability of our results, we conducted rigorous statistical significance analysis on the HTN model. Employing a 5-fold stratified cross-validation strategy, we evaluated model performance on both CodeXGLUE and CWE datasets and performed one-sample t-tests on the cross-validation results for all

Table 4

Ablation experiment results in CodeXGLUE.

Model	AUC	Acc.	Prec.	Recall	F1
Transformer	65.178	59.698	55.629	61.026	58.203
No-Indentation	66.577	61.615	58.293	57.590	57.940
HTN(Ours)	66.635	61.618	56.425	62.269	59.203

Table 5

Ablation experiment results in 10 % CWEDataset.

Model	AUC	Acc.	Prec.	Recall	F1
Transformer	84.900	79.244	77.033	68.562	72.551
No-Indentation	85.042	78.861	74.052	72.634	73.336
HTN(Ours)	85.086	80.156	76.184	73.330	74.730

evaluation metrics (F1-score, accuracy, precision, recall, and ROC-AUC). Statistical analysis results demonstrate that the HTN model significantly outperforms random guessing (baseline = 0.5) across all performance metrics on both datasets, with all p-values being substantially lower than the significance threshold of 0.05 ($p < 0.001$). These findings provide strong statistical evidence for the observed performance improvements, confirming the effectiveness of the proposed enhancement method and ruling out the influence of random factors.

The results indicate that HTN outperforms all baseline models across all evaluation metrics. Specifically, on the CodeXGLUE dataset, our model increased the AUC-ROC value from 0.614 to 0.666, representing a relative improvement of 8.47%. On the CWE dataset, the AUC-ROC value increased from 0.845 to 0.888, reflecting a relative improvement of 4.84%. Consistent improvements were also observed in other evaluation metrics, including F1-score and Precision, across both datasets.

4.2. Contribution of each module to model performance (RQ2)

4.2.1. Ablation experiment

To validate the effectiveness of each module, we conducted a systematic ablation study on the CodeXGLUE dataset. Two sets of ablation experiments were designed:

- Removing the indentation information encoder from the structured representation (No-Indentation).
- Replacing the structured representation module with the original Transformer architecture.

Using a controlled variable approach, we assessed the necessity of the structured representation module and the indentation information encoder design. Tables 4 and 5 presents the quantitative results of these experiments. It is important to note that, given the substantially larger size and higher degree of imbalance in the CWE dataset, we performed 10% random sampling on the balanced CWE dataset during ablation studies to minimize computational overhead. The main purpose of this approach was to investigate whether the performance trends exhibited by HTN on the CodeGLUE dataset are consistent when applied to the CWE dataset.

Empirical results show that removing the indentation information or using an unstructured representation model results in a decrease in prediction performance. This demonstrates that our proposed model effectively learns key information related to source code embedding representations from indentation. Furthermore, mathematical calculations reveal that our structured representation method significantly reduces the model's parameter size for the same input truncation length. This reduction greatly lowers the cost required for model deployment.

4.2.2. Attention-based interpretability analysis

To further validate the effectiveness of the Block Encoder, this study conducted a detailed analysis of the Block Encoder's attention mechanism under experimental configurations where both code length and

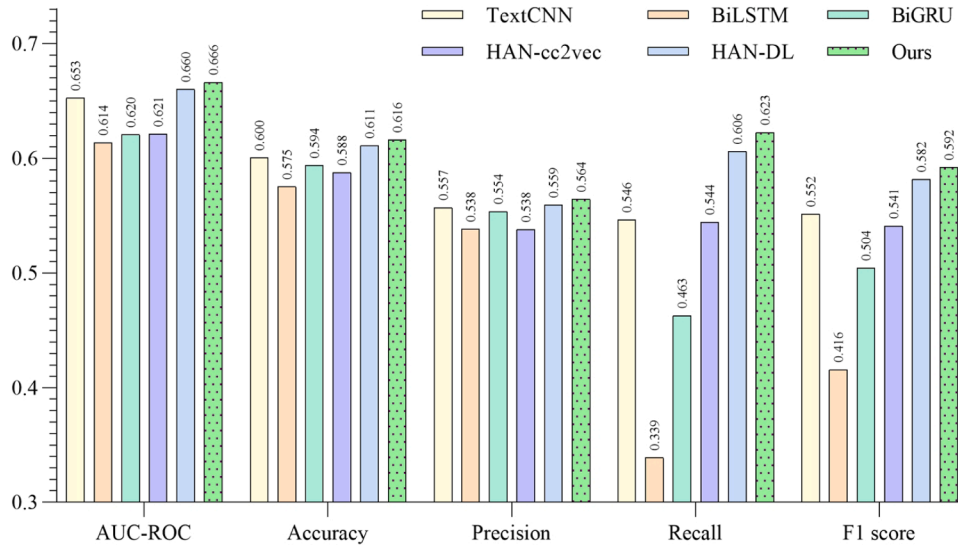


Fig. 5. Evaluation of the approaches on CodeXGLUE dataset.

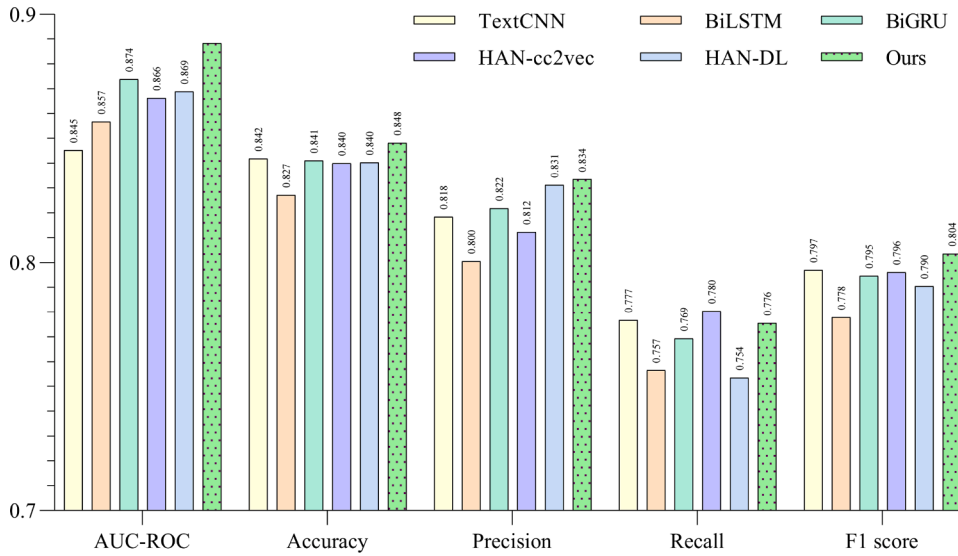


Fig. 6. Evaluation of the approaches on CWE dataset.

```

01. int mma8452_show_int_plus_micros ( char * buf , const int ( * vals ) [ 2 ] ,
02.                                     int n )
03. {
04.     size_t len = 0 ;
05.     while ( n-- > 0 )
06.         len += sprintf ( buf + len , PAGE_SIZE-len , "%d.%06d " ,
07.                         vals [ n ] [ 0 ] , vals [ n ] [ 1 ] ) ;
08.     buf [ len - 1 ] = ' \ n ' ;
09.     return len ;
10. }

```

Fig. 7. A buffer overflow code.

line length were set to 16. This study selected a typical code snippet containing buffer overflow vulnerabilities as the analysis sample (as shown in Fig. 7).

Through static code analysis, this study identified potential security vulnerabilities present in the code snippet: First, in the function declaration section (line 1), the parameter `buf` is defined as a character pointer type, but its buffer boundaries are not explicitly constrained; Second, in the subsequent code execution logic (lines 6 and 8), the program lacks

necessary boundary checking mechanisms when performing write operations on `buf`. Such design flaws in the program may trigger buffer overflow security vulnerabilities.

Fig. 8 presents the attention weight distribution heatmap generated by the Block Encoder module for this code snippet. From the longitudinal analysis results, it can be observed that the Block Encoder allocated significantly higher attention weight values to code segments on lines 1, 6, and 7. This experimental phenomenon demonstrates that the

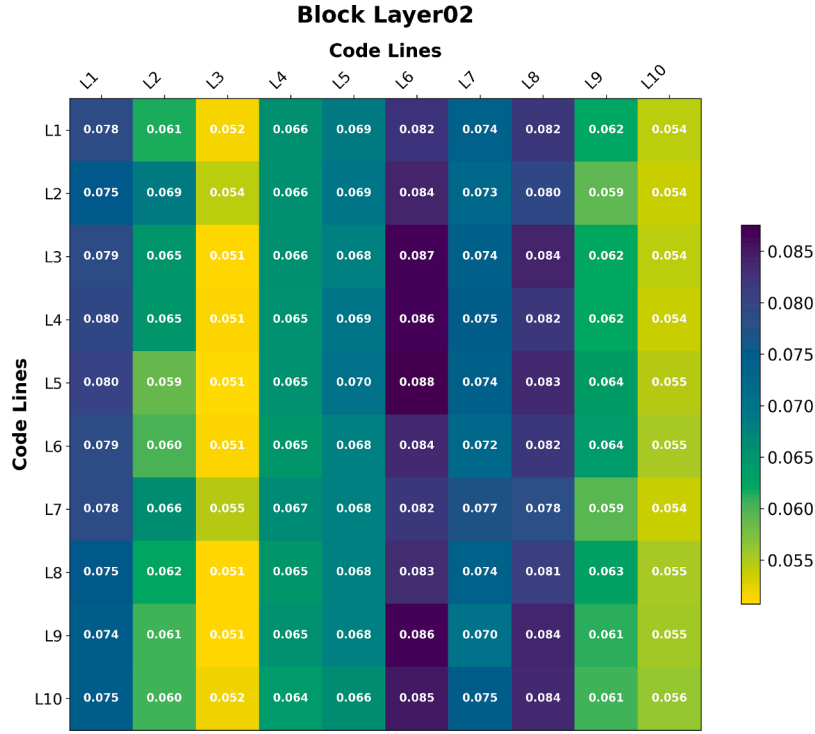


Fig. 8. A buffer overflow code.

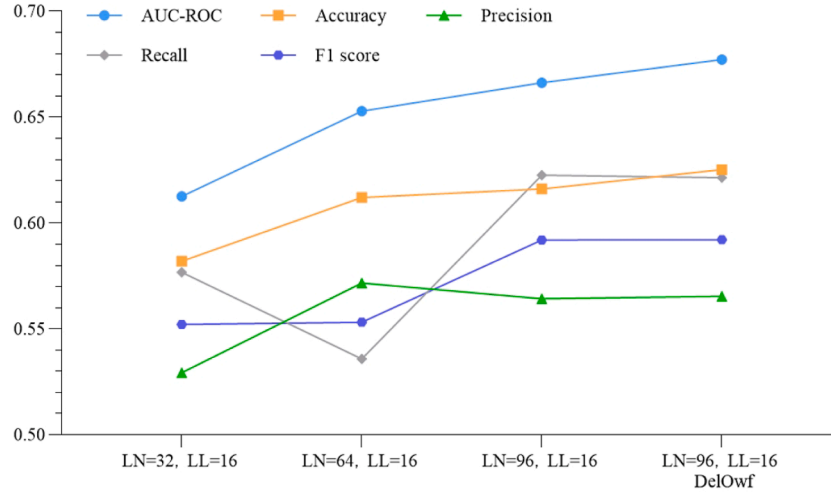


Fig. 9. The impact of code line truncation and outlier data on model performance.(CodeXGLUEDataset).

Block Encoder module can effectively identify and precisely locate critical statements with security risks in the code, thereby validating the effectiveness of this module in software vulnerability detection tasks as well as the interpretability characteristics of the overall HTN architecture.

4.3. Impact of outlier code snippets on model robustness (RQ3)

We experimented on the CodeXGLUE dataset to explore the impact of outlier code snippets. Statistical analysis of the dataset reveals that the average number of lines in the code is 48.7, with only 19.8% of the samples containing more than 64 lines. The average length of individual code lines is 7.7 characters, with 8.4% of the lines exceeding 16 characters. Therefore, we set the maximum line length to 16 characters in our experiment and compared three different line truncation strategies (32/64/96 lines).

In the outlier removal experiment, we minimized the impact on the dataset by removing data points exceeding 1.5 times the maximum number of code lines. Under the four experimental conditions described above, we conducted comparative experiments using the same hyperparameters and recorded the model's performance.

As shown in Figs. 9 and 10, HTN's performance exhibits significant variations under different input length configurations. On the CodeXGLUE dataset, when the line truncation length was increased from 32 to 96 lines, the AUC-ROC improved by 8.65 percentage points (from 0.613 to 0.666). Outlier removal yielded a marginal improvement of 1.65 percentage points (from 0.666 to 0.677). Contribution analysis reveals that the primary performance gain stems from input length optimization (82.4%), while outlier removal accounts for only 17.6% of the improvement. A consistent trend was observed on the 10% sampled CWE dataset. Further analysis indicates that HTN optimizes its learning process based on the inherent information contained within the code

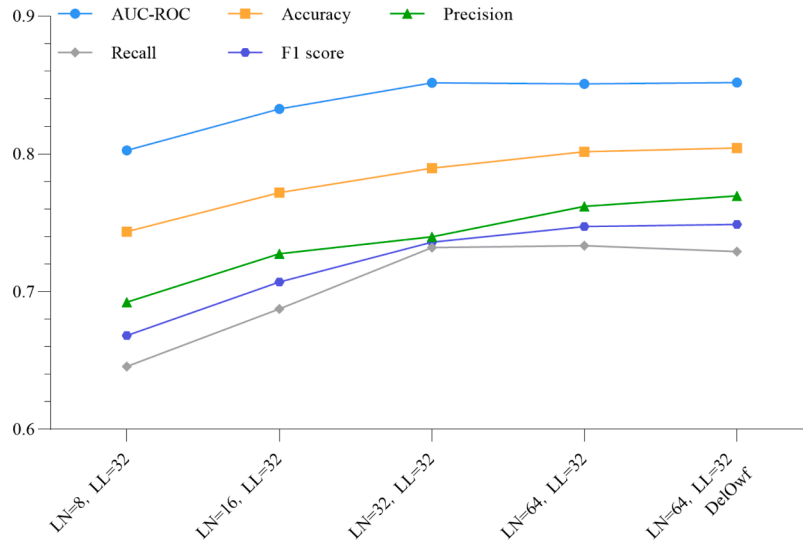


Fig. 10. The impact of code line truncation and outlier data on model performance.(10%CWE Dataset).

Table 6

Performance comparison between HTN and traditional transformer.

Category	Performance Metrics	HTN	Transformer	Improvement
Model Complexity	Total Parameters	276,650,016	274,521,632	+0.8%
	Model Size (MB)	1,055.34	1,047.22	+0.8%
	Mean Inference Latency (ms)	59.51	163.68	-63.6%
Inference Efficiency	Standard Deviation (ms)	0.37	0.15	-
	Throughput (FPS)	16.80	6.11	+175.0%
Memory Utilization	GPU Peak Memory (MB)	1,094.41	1,734.59	-36.9%
	Multi-Head Attention Memory (GB)	3.87	7.50	-48.4%
Computational Efficiency	Total FLOPs (G)	3.26	0.04	-
	Operation Invocations	2600	80	-
Multi-Head Attention Analysis	Average Time per Call (μ s/ms)	80.141(μ s)	12.084(ms)	-99.3%
	CUDA Execution Time (ms)	208.366	966.685	-78.4%

itself. Excessive data truncation leads to the loss of critical semantic and structural information. This finding indirectly corroborates the assertion that HTN can effectively learn key information from code snippets.

4.4. Computational complexity analysis and performance validation (RQ4)

HTN employs positional encoding to transform code representations into a two-dimensional hierarchical structure, thereby overcoming the limitations of traditional one-dimensional sequence processing. From a theoretical perspective, conventional Transformers require constructing an $n \times n$ attention matrix when processing token sequences of length n , whereas HTN's hierarchical design enables the generation of $\sqrt{n} \times \sqrt{n}$ attention matrices through row encoders and block encoders respectively, under the condition that tokens form an ideal square distribution. This design reduces the parameter complexity of attention computation from n^2 to $2n$, achieving a theoretical performance improvement of $\frac{n}{2}$ and reducing computational complexity from quadratic to linear scale.

To validate this theoretical analysis, we configure the maximum token length to 2048, the embedding dimension to 512, the number of attention heads to 4, and the total number of multi-head self-attention layers to 8, and compare HTN with Transformer under these settings. The experimental results in Table 6 demonstrate the significant advantages of HTN: (1) inference time is reduced by 63.6% with a $2.75 \times$ improvement in throughput; (2) multi-head attention memory usage is decreased by 48.4% and peak memory consumption is reduced by 36.9%; (3) global attention is effectively decomposed into efficient local computations; (4) superior performance is achieved under similar model scales, making it particularly suitable for resource-constrained environments.

Despite higher GFLOPs indicating greater computational complexity, HTN achieves faster inference speed through optimized attention mechanisms rather than simple global attention computations. This validates HTN's theoretical advantages in computational cost reduction, particularly in multi-head attention memory optimization, enabling substantial inference cost reductions while maintaining model expressiveness.

5. Threats to validity

In this section, we systematically analyzes potential threats to validity from two dimensions: internal validity and external validity.

5.1. Internal validity threats

5.1.1. Experimental reproducibility bias

Although we strictly followed the implementation of the baseline models described in the original papers and performed cross-validation on all experimental code, two researchers independently implemented key modules. Differences in underlying computations between deep learning frameworks (e.g., PyTorch 1.9 vs. TensorFlow 2.4) may lead to performance fluctuations. We reproduced all comparative experiments under identical software and hardware environments to mitigate this.

5.1.2. Data preprocessing bias

During the reconstruction of the CWE dataset, we used automated scripts and manual sampling checks (randomly checking 5% of the samples) to ensure code parsing accuracy. However, the versioning of GitHub projects may have caused some contextual information to be lost in certain code snippets. We excluded such samples from the training set using the tree-sitter tool to address this threat.

5.2. External validity threats

5.2.1. Dataset limitations

In this study, we evaluated the generalizability of HTN using only two datasets. This may threaten external validity, as HTN could perform differently on datasets not included in our study. However, it is important to note that the two datasets used are widely adopted and contain substantial data, which minimizes this threat. Additionally, we employed the same evaluation metrics as previous research, ensuring comparability.

5.2.2. Hyperparameter sensitivity

The learning rate range (1e-5 to 1e-8) used for baseline models may not have been optimal. However, we verified this through grid search (with a step size of 0.5 orders of magnitude) and an early stopping strategy (patience=20) to ensure all models converged to a stable state.

This study ensures the reliability and reproducibility of experimental results through multidimensional threat analysis and systematic mitigation strategies. Future work will expand multilingual support and integrate real-time debugging information.

6. Conclusion and future works

In this paper, we propose a novel feature for representing the structural information of source code and introduce the HTN model, which generates embeddings for code using a hierarchical self-attention network. The HTN model leverages self-attention mechanisms and Transformer-based positional encoding to capture key structural features related to code defects, producing more accurate code vector representations for defect prediction. We evaluate HTN on two datasets, demonstrating its effectiveness in defect prediction compared to other models. Ablation experiments highlight the importance of each component in the model, showing that omitting any part negatively impacts performance.

Future work will focus on extending HTN to other source code-related tasks, further evaluating the utility of the proposed structural features, and developing pre-training tasks to improve the model's performance and generalization.

CRedit authorship contribution statement

Hongdou He: Writing – review & editing, Resources, Formal analysis; **Huazhi Xu:** Writing – original draft, Methodology; **Guyu Zhao:** Supervision, Funding acquisition, Conceptualization; **Guoyan Huang:** Supervision; **Bowen Zhao:** Investigation.

Data availability

The data that has been used is confidential.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work is funded by the [National Natural Science Foundation of China](#) (No. 62302428); the Hebei Natural Science Foundation (No. F2024203085); and the Science Research Project of Hebei Education Department (No. BJ2025012). The authors are grateful to valuable comments and suggestions of the reviewers.

References

- Abdu, A., et al., 2024. Software defect prediction based on deep representation learning of source code from contextual syntax and semantic graph. *IEEE Trans. Reliab.* 73 (2), 820–834.
- Chakraborty, S., et al., 2021. Deep learning based vulnerability detection: are we there yet? *IEEE Trans. Software Eng.* 48 (9), 3280–3296.
- Cheng, X., et al., 2022. Path-sensitive code embedding via contrastive learning for software vulnerability detection. In: *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*.
- Ciborowska, A., Damevski, K., 2022. Fast changeset-based bug localization with BERT. In: *Proceedings of the 44th International Conference on Software Engineering*.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al., 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv:2010.11929*.
- Feng, Z., et al., 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv:2002.08155*.
- Fu, M., Tantithamthavorn, C., 2022. Linevul: a transformer-based line-level vulnerability prediction. In: *Proceedings of the 19th International Conference on Mining Software Repositories*.
- Gesi, J., Li, J., Ahmed, I., 2021. An empirical examination of the impact of bias on just-in-time defect prediction. In: *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*.
- Guo, D., et al., 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv:2009.08366*.
- Guo, D., et al., 2022. Unixcoder: Unified cross-modal pre-training for code representation. *arXiv:2203.03850*.
- Hin, D., et al., 2022. LineVD: statement-level vulnerability detection using graph neural networks. In: *Proceedings of the 19th International Conference on Mining Software Repositories*.
- Hoang, T., et al., 2019. Deepjit: an end-to-end deep learning framework for just-in-time defect prediction. In: *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE.
- Hoang, T., et al., 2020. Cc2vec: distributed representations of code changes. In: *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*.
- Kamei, Y., Shihab, E., Adams, B., et al., 2012. A large-scale empirical study of just-in-time quality assurance. *IEEE Trans. Software Eng.* 39 (6), 757–773.
- Le, T. H.M., et al., 2021. DeepCVA: automated commit-level vulnerability assessment with deep multi-task learning. In: *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE.
- Li, C., et al., 2020. On deep unsupervised active learning. *arXiv:2007.13959*.
- Lu, S., et al., 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv:2102.04664*.
- Mahbub, P., Rahman, M.M., 2024. Predicting line-level defects by capturing code contexts with hierarchical transformers. In: *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE.
- Mou, L., et al., 2016. Convolutional neural networks over tree structures for programming language processing. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 30.
- Nguyen, V.-A., et al., 2022. ReGVD: revisiting graph neural networks for vulnerability detection. In: *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*.
- Ni, Z., et al., 2020. Analyzing bug fix for automatic bug cause classification. *J. Syst. Softw.* 163, 110538.
- Niu, C., et al., 2022. Spt-code: sequence-to-sequence pre-training for learning source code representations. In: *Proceedings of the 44th International Conference on Software Engineering*.
- Pascarella, L., Palomba, F., Bacchelli, A., 2019. Fine-grained just-in-time defect prediction. *J. Syst. Softw.* 150, 22–36.
- Popescu, M.C., Balas, V.E., Perescu-Popescu, L., et al., 2009. Multilayer perceptron and neural networks. *WSEAS Trans. Circuits and Syst.* 8 (7), 579–588.

- Pornprasit, C., Tantithamthavorn, C.K., 2022. Deeplinedp: towards a deep learning approach for line-level defect prediction. *IEEE Trans. Software Eng.* 49 (1), 84–98.
- Russell, R., et al., 2018. Automated vulnerability detection in source code using deep representation learning. In: 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA). IEEE.
- Tang, W., Tang, M., Ban, M., et al., 2023. Csgvd: a deep learning approach combining sequence and graph embedding for source code vulnerability detection. *J. Syst. Softw.* 199, 111623.
- Vaswani, A., et al., 2017. Attention is all you need. In: *Advances in Neural Information Processing Systems*. Vol. 30.
- Wartschinski, L., et al., 2022. Vudenc: vulnerability detection with deep learning on a natural codebase for python. *Inf. Softw. Technol.* 144, 106809.
- Wu, Y., et al., 2022. Vulcnn: an image-inspired scalable vulnerability detection system. In: *Proceedings of the 44th International Conference on Software Engineering*.
- Zhang, J., et al., 2019. A novel neural source code representation based on abstract syntax tree. In: 2019IEEE/ACM 41st International Conference on Software Engineering (ICSE). IEEE.
- Zhang, Q., Wu, B., 2020. Software defect prediction via transformer. In: 2020 IEEE 4th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC). IEEE.
- Zhou, C., et al., 2022. Software defect prediction with semantic and structural information of codes based on graph neural networks. *Inf Softw Technol* 152, 107057.
- Zhou, Y., et al., 2019. Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: *Advances in Neural Information Processing Systems*. Vol. 32.
- Zhuang, Y., et al., 2021. Smart contract vulnerability detection using graph neural networks. In: *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*.